

```

where NEW.tag ilike 'f%'
DO INSTEAD insert into f_tags(pk,tag,parent)values (NEW.pk,NEW.
tag,NEW.parent);

```

This time, in the rule, we simply told PostgreSQL that every time a record is inserted with a tag value that starts with the letter f, or the capital letter F, it must be moved into the f_tags table.

3. Now let's perform this query:

```

forumdb=> insert into tags (tag) values ('Fedora Linux');
INSERT 0 0

```

Already from the answer, INSERT 0 0, we can guess that nothing has been inserted into the tags table.

4. Now, we will perform this statement:

```

forumdb=> select * from tags;
pk |      tag      | parent
----+-----+-----
 1 | Operating Systems |
 2 | Linux          |      1
 3 | Ubuntu         |      2
 4 | OpenBSD        |
(4 rows)

```

5. As we can see in the preceding snippet, the value Fedora Linux does not appear in the tags table, and in the f_tags table, we will have the following:

```

forumdb=> select * from f_tags ;
pk |      tag      | parent
----+-----+-----
 6 | Fedora Linux |
(1 row)

```

The rule that we defined made sure that the record was not inserted into the tags table but was inserted into the f_tags table.

6. As the last example of the INSERT rule, suppose we want nothing to be inserted every time a record is inserted with the tag field that starts with the letter R or r.

As we did before, let's perform the rule:

```
forumdb=> create or replace rule r_tags3
as on INSERT to tags
where NEW.tag ilike 'r%'
DO INSTEAD NOTHING;
CREATE RULE
```

7. This time, we've said to PostgreSQL that every time the tags table receives a record with the field tag that starts with the letter r or R, this record should not be considered. Let's try what we've said:

```
forumdb=> insert into tags (tag) values ('Red Hat Linux');
INSERT 0 0
```

8. Even now, we have INSERT 0 0 as the answer from the server, and we can check that the record has not been inserted in any table:

```
forumdb=> select pk,tag,parent,'tags' as tablename
from tags
union all
select pk,tag,parent,'f_tags' as tablename
from f_tags
order by tablename, tag;
pk | tag | parent | tablename
----+-----+-----+-----
6 | Fedora Linux | | f_tags
2 | Linux | 1 | tags
4 | OpenBSD | | tags
1 | Operating Systems | | tags
3 | Ubuntu | 2 | tags
(5 rows)
```

As we can see, the record does not appear in any table. In the preceding query, we used UNION ALL. This includes the results of the two queries. The important thing is that the field types must be compatible with each other.

Rules on DELETE/UPDATE

In the previous section, we looked at how to use rules on INSERT events. In this section, we will see how to use rules on DELETE and UPDATE events.

We will now look at a complete example of how to use the rules, starting from the concepts described above.

The goal we want to reach is described in the following steps:

1. Create a table called `new_tags` equal to the `tags` table; this table will help us to have a clean environment where we can do our tests.
2. Create two tables: a table called `new_a_tags` for a copy of all records with the tags that start with the letter `a`, and a table called `new_b_tags` for a copy of all records with the tags that start with the letter `b`.
3. Create all the `INSERT/DELETE/UPDATE` rules that make everything work.

Let's begin.

Creating the `new_tags` table

The first step is to create a new `new_tags` table. We will create this table based on the existing `tags` table:

```
forumdb=> create table new_tags as select * from tags limit 0;
SELECT 0
```

```
forumdb=# \d new_tags
```

```
          Table "public.new_tags"
  Column | Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
pk      | integer   |           |          |
tag     | text      |           |          |
parent  | integer   |           |          |
```

The preceding statement copies the structure of the fields of the `tags` table into the `new_tags` table, but it does not copy the constraints or any indices. Now we have to create the primary key constraint on the new table:

```
forumdb=> alter table new_tags alter pk set not null ;
ALTER TABLE
forumdb=> alter table new_tags add constraint new_tags_pk primary key
(pk);
ALTER TABLE
forumdb=# \d new_tags
```

```

Table "public.new_tags"
Column | Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
pk      | integer   |           | not null |
tag     | text      |           |          |
parent  | integer   |           |          |
Indexes:
    "new_tags_pk" PRIMARY KEY, btree (pk)

```

With this, *step 1* is complete.

Creating two tables

Similar to what we just did, let's create `new_a_tags` and `new_b_tags` tables. For the `new_a_tags` table, we will have the following:

```

forumdb=> create table new_a_tags as select * from tags limit 0;
SELECT 0
forumdb=> alter table new_a_tags alter pk set not null ;
ALTER TABLE
forumdb=> alter table new_a_tags add constraint new_a_tags_pk primary key
(pk);
ALTER TABLE
forumdb=> \d new_a_tags

Table "forum.new_a_tags"
Column | Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
pk      | integer   |           | not null |
tag     | text      |           |          |
parent  | integer   |           |          |
Indexes:
    "new_a_tags_pk" PRIMARY KEY, btree (pk)

```

In the same way, we will create the `new_b_tags` table:

```

forumdb=> create table new_b_tags as select * from tags limit 0;
SELECT 0
forumdb=> alter table new_b_tags alter pk set not null ;
ALTER TABLE

```

```

forumdb=> alter table new_b_tags add constraint new_b_tags_pk primary key
(pk);
ALTER TABLE
forumdb=> \d new_b_tags
          Table "forum.new_b_tags"
  Column | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
 pk      | integer |           | not null |
 tag     | text    |           |          |
 parent  | integer |           |          |
Indexes:
    "new_b_tags_pk" PRIMARY KEY, btree (pk)

```

Step 2 is now complete, and we have everything we need to start our complete example.

Managing rules on INSERT, DELETE, and UPDATE events

The goal we want to achieve is shown in the following figure:

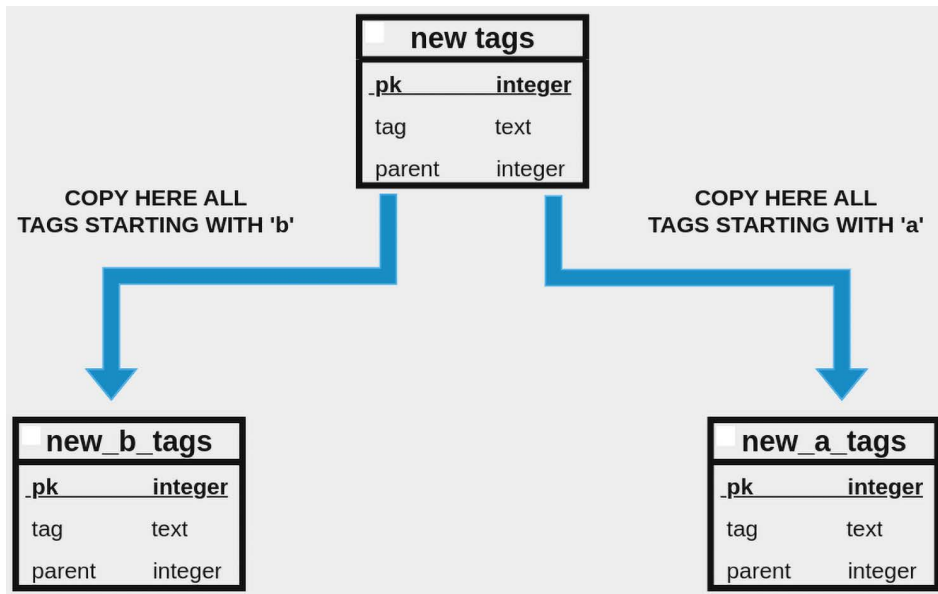


Figure 8.1: Managing rules

We want all tags starting with the letter a to be stored in the new_tags table and also copied to the new_a_tags table, and we want the same for tags that begin with the letter b.

We have to manage rules for INSERT, DELETE, and UPDATE events in the following ways:

- INSERT rules must recognize all tags starting with the letters a or b and copy those records into their respective tables – new_a_tags and new_b_tags.
- DELETE rules must recognize all the tags starting with the letters a or b and delete those records in the respective tables – new_a_tags and new_b_tags.
- UPDATE rules must recognize all the tags that begin with the letters a or b, and if a record changes its tag, the rule must check whether the record should be copied or deleted in the new_a_tags and new_b_tags tables.

INSERT rules

Let's start by creating two INSERT rules:

```

forumdb=# create or replace rule r_new_tags_insert_a as on INSERT to new_
tags where NEW.tag like 'a%' DO ALSO insert into new_a_tags(pk,tag,parent)
values (NEW.pk,NEW.tag,NEW.parent);
CREATE RULE

forumdb=# create or replace rule r_new_tags_insert_b as on INSERT to new_
tags where NEW.tag like 'b%' DO ALSO insert into new_b_tags(pk,tag,parent)
values (NEW.pk,NEW.tag,NEW.parent);
CREATE RULE

```

As we can see, the new_tags table now has two new rules:

```

forumdb=# \d new_tags
               Table "forum.new_tags"
  Column |  Type  | Collation | Nullable | Default
-----+-----+-----+-----+-----
 pk      | integer |           | not null |
 tag     | text    |           |          |
 parent  | integer |           |          |
Indexes:
    "new_tags_pk" PRIMARY KEY, btree (pk)
Rules:
    r_new_tags_insert_a AS
    ON INSERT TO new_tags
    WHERE new.tag ~~ 'a% '::text DO  INSERT INTO new_a_tags (pk, tag,
parent)

```

```
VALUES (new.pk, new.tag, new.parent)
  r_new_tags_insert_b AS
  ON INSERT TO new_tags
  WHERE new.tag ~~ 'b%':text DO INSERT INTO new_b_tags (pk, tag,
parent)
VALUES (new.pk, new.tag, new.parent)
```

To check whether the rules work, let's insert some data:

```
forumdb=> insert into new_tags values(1,'linux',NULL);
INSERT 0 1
forumdb=> insert into new_tags values(2,'alpine linux',1);
INSERT 0 1

forumdb=> insert into new_tags values(3,'bsd unix',NULL);
INSERT 0 1
```

Then let's check the parent table:

```
forumdb=> select * from new_tags ;
pk |      tag      | parent
----+-----+-----
  1 | linux         |
  2 | alpine linux  |      1
  3 | bsd unix      |
(3 rows)
```

Now let's see what is in the table_a child table:

```
forumdb=> select * from new_a_tags ;
pk |      tag      | parent
----+-----+-----
  2 | alpine linux  |      1
(1 row)
```

And what's in the table_b child table:

```
forumdb=> select * from new_b_tags ;
pk |      tag      | parent
----+-----+-----
```

```
3 | bsd unix |
(1 row)
```

We can see that the two rules work.

DELETE rules

Now let's create the DELETE rules. We need rules that, if a record is deleted from the `new_tags` table and it begins with the letter a or b, its copy in the `new_a_tags` and `new_b_tags` table must also be deleted. For all the records that start with the letter a, we need this rule:

```
forumdb=> create or replace rule r_new_tags_delete_a as on delete to new_
tags where OLD.tag like 'a%' DO ALSO delete from new_a_tags where pk=OLD.
pk;
CREATE RULE
```

Similarly, we need this rule for records beginning with the letter b:

```
forumdb=> create or replace rule r_new_tags_delete_b as on delete to new_
tags where OLD.tag like 'b%' DO ALSO delete from new_b_tags where pk=OLD.
pk;
CREATE RULE
```

The current situation of the `new_tags` table is as follows:

```
forumdb=> \d new_tags

          Table "forum.new_tags"
  Column |  Type  | Collation | Nullable | Default
-----+-----+-----+-----+-----
 pk      | integer |           | not null |
 tag     | text   |           |          |
 parent  | integer |           |          |
Indexes:
    "new_tags_pk" PRIMARY KEY, btree (pk)
Rules:
    r_new_tags_delete_a AS
    ON DELETE TO new_tags
    WHERE old.tag ~~ 'a% '::text DO  DELETE FROM new_a_tags
    WHERE new_a_tags.pk = old.pk
    r_new_tags_delete_b AS
    ON DELETE TO new_tags
```

```

WHERE old.tag ~~ 'b% '::text DO DELETE FROM new_b_tags
WHERE new_b_tags.pk = old.pk
  r_new_tags_insert_a AS
  ON INSERT TO new_tags
  WHERE new.tag ~~ 'a% '::text DO INSERT INTO new_a_tags (pk, tag,
parent)
VALUES (new.pk, new.tag, new.parent)
  r_new_tags_insert_b AS
  ON INSERT TO new_tags
  WHERE new.tag ~~ 'b% '::text DO INSERT INTO new_b_tags (pk, tag,
parent)
VALUES (new.pk, new.tag, new.parent)

```

Let's test whether the two new rules work:

```

forumdb=> delete from new_tags where tag = 'alpine linux';
DELETE 1
forumdb=> delete from new_tags where tag = 'bsd unix';
DELETE 1

forumdb=> select * from new_tags ;
pk | tag | parent
----+-----+-----
  1 | linux |
(1 row)

forumdb=> select * from new_a_tags ;
pk | tag | parent
----+-----+-----
(0 rows)

forumdb=> select * from new_b_tags ;
pk | tag | parent
----+-----+-----
(0 rows)

```

We can see from this that the new rules work.

UPDATE rules

Now we need to introduce a rule that checks whether a tag is updated with a word that starts with a or b. The simple way to do this is to first create a function that conducts this check and then create a rule based on that function. Let's start by creating the function:

```
forumbd=> create or replace function move_record (p_pk integer, p_tag
text, p_parent integer,p_old_pk integer,p_old_tag text ) returns void
language plpgsql as
$$
BEGIN
    if left(lower(p_tag),1) in ('a','b') THEN
        delete from new_tags where pk = p_old_pk;
        insert into new_tags values(p_pk,p_tag,p_parent);
    end if;
END;
$$;
CREATE FUNCTION
```

This function takes five parameters as input; the first three parameters are the NEW values that arrive from the update, and the last two parameters are the OLD values of the record that are present in the record. The function checks if the record in the table starts with the letter a or b, and it deletes the old record and inserts the new record.

So, finally, the rule is as follows:

```
forumbd=> create or replace rule r_new_tags_update_a as on UPDATE to new_
tags DO ALSO select move_record(NEW.pk,NEW.tag,NEW.parent,OLD.pk,OLD.tag);
CREATE RULE
```

The rule calls the function if there is an update. Let's see if this rule works:

```
forumbd=> update new_tags set tag='alpine linux' where tag='linux';
move_record
-----

(1 row)
UPDATE 0

forumbd=> select * from new_a_tags ;
```